

→ Week 3 Module:-

Q1.

```
#include <stdio.h>
```

```
void merge (int a[], int l, int m, int r)
```

```
{ int i = l, j = m + 1, k = 0, temp[100];
```

```
while (i <= m && j <= r)
```

```
temp[k++] = (a[i] < a[j]) ? a[i++] : a[j++];
```

```
while (i <= m) temp[k++] = a[i++];
```

```
while (j <= r) temp[k++] = a[j++];
```

```
for (i = l, k = 0; i <= r; i++)
```

```
a[i] = temp[k++];
```

```
}
```

```
void mergesort (int a[], int l, int r)
```

```
{ if (l < r)
```

```
{ int m = (l + r) / 2;
```

```
mergesort (a, l, m);
```

```
mergesort (a, m + 1, r);
```

```
mergesort (a, l, m, r);
```

```
}
```

```
}
```

```
int main ()
```

```
{ int n, i, a[100];
```

```
printf ("Enter number of students: ");
```

```
scanf ("%d", &n);
```

```
printf ("Enter marks of students: \n");
```

```
for (i = 0; i < n; i++)
```

```
scanf ("%d", &a[i]);
```

```
mergesort (a, 0, n - 1);
```

```
for (i = 0; i < n; i++)
```

```
printf ("%d", a[i]);
```

```
return 0;
```

Output :-

Enter number of students : 6

Enter marks of students :

78

45

92

34

67

01

34 45 67 78 01 92

Explanation :-

- Read the no. of students and their marks
- Divide the array into smaller parts
- Sort each part recursively
- Merge the sorted parts in ascending order
- Print the sorted marks.

```
*1M.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
*1M.c X
1 #include <stdio.h>
2 void merge(int a[], int l, int m, int r)
3 {   int i=l, j=m+1, k=0, temp[100];
4     while(i<=m && j<=r)
5         temp[k++]=(a[i]<a[j])?a[i++]:a[j++];
6     while(i<=m) temp[k++]=a[i++];
7     while(j<=r) temp[k++]=a[j++];
8     for(i=l, k=0; i<=r; i++)
9         a[i]=temp[k++];
10 }
11 void mergesort(int a[], int l, int r)
12 {   if(l<r)
13     {   int m=(l+r)/2;
14         mergesort(a, l, m);
15         mergesort(a, m+1, r);
16         merge(a, l, m, r);
17     }
18 }
19 int main()
20 {   int n, i, a[100];
21     printf("Enter number of students: ");
22     scanf("%d", &n);
23     printf("Enter marks of students:\n");
24     for(i=0; i<n; i++)
25         scanf("%d", &a[i]);
26     mergesort(a, 0, n-1);
27     for(i=0; i<n; i++)
28         printf("%d ", a[i]);
29     return 0;
30 }
31
```

```
"C:\c (PPS)\Module_3\1M.exe" X
+ v
Enter number of students: 6
Enter marks of students:
78
45
92
34
67
81
34 45 67 78 81 92
Process returned 0 (0x0)   execution time : 20.387 s
Press any key to continue.
```

```
#include <stdio.h>
```

```
void merge (int a[], int l, int m, int r)
```

```
{ int i = l, j = m + 1, k = 0, b[100];
```

```
while (i <= m && j <= r)
```

```
{ if (a[i] < a[j])
```

```
    b[k++] = a[i++];
```

```
    else
```

```
        b[k++] = a[j++];
```

```
while (i <= m) b[k++] = a[i++];
```

```
while (j <= r) b[k++] = a[j++];
```

```
for (i = l, k = 0; i <= r; i++)
```

```
    a[i] = b[k++];
```

```
void sort (int a[], int l, int r)
```

```
{ if (l < r)
```

```
{ int m = (l + r) / 2;
```

```
  sort (a, l, m);
```

```
  sort (a, m + 1, r);
```

```
  merge (a, l, m, r);
```

```
int main()
```

```
{ int n, a[100], i;
```

```
printf("Enter number of products: ");
```

```
scanf("%d", &n);
```

```
printf("Enter product prices: \n");
```

```
for (i = 0; i < n; i++)
```

```
    scanf("%d", &a[i]);
```

```
sort (a, 0, n - 1);
```

```
for (i = 0; i < n; i++)
```

```
    printf("%d", a[i]);
```

```
return 0;
```


Output:-

Enter number of products : 5

Enter product prices :

2500

1500

4000

3000

1000

1000 1500 2500 3000 4000

Explanation :-

→ Read the no. of product prices.

→ Divide the array into two halves.

→ Sort each half using Merge Sort.

→ Merge the sorted halves.

→ Print the prices in ascending order.

```
2M.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
2M.c
#include <stdio.h>
void merge(int a[],int l,int m,int r)
{
    int i=l,j=m+1,k=0,b[100];
    while(i<=m&& j<=r)
    {
        if(a[i]<a[j])
            b[k++]=a[i++];
        else
            b[k++]=a[j++];
    }
    while(i<=m) b[k++]=a[i++];
    while(j<=r) b[k++]=a[j++];
    for(i=l,k=0;i<=r;i++)
        a[i]=b[k++];
}
void sort(int a[],int l,int r)
{
    if(l<r)
    {
        int m=(l+r)/2;
        sort(a,l,m);
        sort(a,m+1,r);
        merge(a,l,m,r);
    }
}
int main()
{
    int n,a[100],i;
    printf("Enter number of products: ");
    scanf("%d",&n);
    printf("Enter product prices:\n");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    sort(a,0,n-1);
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
    return 0;
}
```

```
"C:\c (PPS)\Module_3\2M.exe"
Enter number of products: 5
Enter product prices:
2500
1500
4000
3000
1000
1000 1500 2500 3000 4000
Process returned 0 (0x0)    execution time : 30.458 s
Press any key to continue.
```

Q3.

```
#include <stdio.h>
```

```
void merge (int a[], int l, int m, int r)
```

```
{ int i = l, j = m+1, k = 0, t[100];
```

```
while (i <= m && j <= r)
```

```
{ if (a[i] < a[j])
```

```
    t[k++] = a[i++];
```

```
    else
```

```
    t[k++] = a[j++];
```

```
while (i <= m)
```

```
    t[k++] = a[i++];
```

```
while (j <= r)
```

```
    t[k++] = a[j++];
```

```
for (i = l; i <= r; i++)
```

```
    a[i] = t[k++];
```

```
}
```

```
void mergesort (int a[], int l, int r)
```

```
{ if (l < r)
```

```
{ int m = (l+r)/2;
```

```
    mergesort (a, l, m);
```

```
    mergesort (a, m+1, r);
```

```
    merge (a, l, m, r);
```

```
}
```

```
}
```

```

int main ()
{
    int n, a[100], i;
    printf("Number of salaries: ");
    scanf("%d", &n);
    printf("\n Input salaries: \n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    mergesort(a, 0, n-1);
    printf("\n Sorted salaries: \n");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n Least salary: %d", a[0]);
    printf("\n Highest salary: %d", a[n-1]);
    printf("\n Difference: %d", a[n-1] - a[0]);
    return 0;
}

```

Output :-

Number of salaries: 7

Input salaries:

45000

25000

70000

50000

30000

90000

60000

Sorted salaries:

25000 30000 45000 50000 60000 70000 90000

Least salary: 25000

Highest salary: 90000

Difference: 65000

Explanation :-

- Read all employee salaries
- Sort the salaries using merge sort
- Display the sorted salaries
- Display the least and highest salary
- Calculate and print the difference between highest and least.


```
3M.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
3M.c X
1 #include <stdio.h>
2 void merge(int a[],int l,int m,int r)
3 {   int i=l,j=m+1,k=0,t[100];
4     while(i<=m&& j<=r)
5     {   if(a[i]<a[j])
6         t[k++]=a[i++];
7         else
8             t[k++]=a[j++];
9     }
10    while(i<=m)
11        t[k++]=a[i++];
12    while(j<=r)
13        t[k++]=a[j++];
14    for(i=l,k=0;i<=r;i++)
15        a[i]=t[k++];
16 }
17 void mergeSort(int a[],int l,int r)
18 {   if(l<r)
19     {   int m=(l+r)/2;
20         mergeSort(a,l,m);
21         mergeSort(a,m+1,r);
22         merge(a,l,m,r);
23     }
24 }
25 int main()
26 {   int n,a[100],i;
27     printf("Number of salaries: ");
28     scanf("%d",&n);
29     printf("\nInput salaries:\n");
30     for(i=0;i<n;i++)
31         scanf("%d",&a[i]);
32     mergeSort(a,0,n-1);
33     printf("\nSorted Salaries:\n");
34     for(i=0;i<n;i++)
35         printf("%d ",a[i]);
36     printf("\nLowest Salary: %d",a[0]);
37     printf("\nHighest Salary: %d",a[n-1]);
38     printf("\nDifference: %d",a[n-1]-a[0]);
39     return 0;
40 }
41 }
```

Number of salaries: 7

Input salaries:

45000
25000
70000
50000
30000
90000
60000

Sorted Salaries:

25000 30000 45000 50000 60000 70000 90000

Lowest Salary: 25000

Highest Salary: 90000

Difference: 65000

Process returned 0 (0x0) execution time : 246.215 s

Press any key to continue.

Q 4.

```
#include <stdio.h>
```

```
void quick (int a[], int l, int h)
```

```
{ if (l >= h)
```

```
    return;
```

```
    int i = l, j = h, pivot = a[l], temp;
```

```
    while (i < j)
```

```
    { while (a[i] <= pivot && i < h)
```

```
        i++;
```

```
        while (a[j] > pivot)
```

```
            j--;
```

```
        if (i < j)
```

```
        { temp = a[i];
```

```
          a[i] = a[j];
```

```
          a[j] = temp;
```

```
        }
```

```
    } temp = a[l];
```

```
    a[l] = a[j];
```

```
    a[j] = temp;
```

```
    quick (a, l, j-1);
```

```
    quick (a, j+1, h);
```

```
}
```

```
int main()
```

```
{ int n, a[100], i;
```

```
    printf ("Enter number of time records:");
```

```
    scanf ("%d", &n);
```

```
    printf ("\nEnter time records: \n");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf ("%d", &a[i]);
```

```
    quick (a, 0, n-1);
```

```
    for (i = 0; i < n; i++)
```

```
        printf ("%d", a[i]); return 0;
```

Output :-

Enter number of time records: 6

Enter time records:

55

49

62

58

45

51

45 49 51 55 58 62

Explanation :-

- Read the race timings.
- select the first element as the pivot
- Place smaller elements on the left and larger elements on the right
- Repeat the process recursively.
- Print the sorted timings.

```
1Q.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
1Q.c X
main(): int
1 #include <stdio.h>
2 void quick(int a[], int l, int h)
3 { if(l >= h)
4     return;
5     int i = l, j = h, pivot = a[l], temp;
6     while(i < j)
7     { while(a[i] <= pivot && i < h)
8         i++;
9         while(a[j] > pivot)
10             j--;
11         if(i < j)
12         { temp = a[i];
13           a[i] = a[j];
14           a[j] = temp;
15         }
16     }
17     temp = a[l];
18     a[l] = a[j];
19     a[j] = temp;
20     quick(a, l, j - 1);
21     quick(a, j + 1, h);
22 }
23 int main()
24 { int n, a[100], i;
25   printf("Enter number of time records: ");
26   scanf("%d", &n);
27   printf("\nEnter time records:\n");
28   for(i = 0; i < n; i++)
29       scanf("%d", &a[i]);
30   quick(a, 0, n - 1);
31   for(i = 0; i < n; i++)
32       printf("%d ", a[i]);
33   return 0;
34 }
```

```
"C:\c (PPS)\Module_3\1Q.exe" X
Enter number of time records: 6
Enter time records:
55
49
62
58
45
51
45 49 51 55 58 62
Process returned 0 (0x0)   execution time : 24.090 s
Press any key to continue.
```


Q5.

```
#include <stdio.h>
```

```
void sort (int a[], int l, int h)
```

```
{ if (l >= h)
```

```
    return;
```

```
    int i = l, j = h, p = a[l], t;
```

```
    while (i < j)
```

```
    { while (a[i] <= p && i < h)
```

```
        i++;
```

```
        while (a[j] > p)
```

```
            j--;
```

```
        if (i < j)
```

```
        { t = a[i];
```

```
          a[i] = a[j];
```

```
          a[j] = t;
```

```
        }
```

```
    } t = a[l];
```

```
    a[l] = a[j];
```

```
    a[j] = t;
```

```
    sort(a, l, j-1);
```

```
    sort(a, j+1, h);
```

```
}
```

```
int main ( )
```

```
{ int n, a[100], i;
```

```
    printf("Enter number of book IDs : ");
```

```
    scanf("%d", &n);
```

```
    printf("\n Enter IDs : \n");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf("%d", &a[i]);
```

```
    sort(a, 0, n-1);
```

```
    for (i = 0; i < n; i++)
```

```
        printf("%d", a[i]);
```

```
    return 0;
```

Output :-

Enter number of book IDs: 5

Enter IDs:

1025

1001

1050

1010

1005

1001 1005 1010 1025 1050

Explanation :-

→ Read the book IDs

→ Use the first element as the pivot

→ Rearrange the array around the pivot

→ Recursively sort the left and right parts

→ Print the sorted IDs

```
2Qc - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
2Qc X
1 #include <stdio.h>
2 void sort(int a[], int l, int h)
3 { if(l >= h)
4     return;
5     int i = l, j = h, p = a[l], t;
6     while(i < j)
7     { while(a[i] <= p && i < h)
8         i++;
9         while(a[j] > p)
10             j--;
11         if(i < j)
12         { t = a[i];
13           a[i] = a[j];
14           a[j] = t;
15         }
16     }
17     t = a[l];
18     a[l] = a[j];
19     a[j] = t;
20     sort(a, l, j - 1);
21     sort(a, j + 1, h);
22 }
23 int main()
24 { int n, a[100], i;
25   printf("Enter number of book IDs: ");
26   scanf("%d", &n);
27   printf("\nEnter IDs:\n");
28   for(i = 0; i < n; i++)
29       scanf("%d", &a[i]);
30   sort(a, 0, n - 1);
31   for(i = 0; i < n; i++)
32       printf("%d ", a[i]);
33   return 0;
34 }
```

```
"C:\c (PPS)\Module_3\2Q.exe"
Enter number of book IDs: 5
Enter IDs:
1025
1001
1050
1010
1005
1001 1005 1010 1025 1050
Process returned 0 (0x0)   execution time : 42.401 s
Press any key to continue.
```


#include <stdio.h>

void quick (int a[], int l, int h)

{ if (l >= h)

return;

int i = l, j = h, p = a[l], t;

while (i < j)

{ while (a[i] <= p && i < h)

i++;

while (a[j] > p)

j--;

if (i < j)

{ t = a[i];

a[i] = a[j];

a[j] = t;

t = a[l];

a[l] = a[j];

a[j] = t;

quick (a, l, j-1);

quick (a, j+1, h);

int main()

```
{ int n, a[100], i, sum = 0;
  printf("Enter number of salaries: ");
  scanf("%d", &n);
  printf("\nEnter salaries: \n");
  for(i = 0; i < n; i++)
    scanf("%d", &a[i]);
  quick(a, 0, n-1);
  printf("Sorted sales: \n");
  for(i = n-1; i >= 0; i--)
    printf("%d", a[i]);
  printf("\n\nTop 3: %d %d %d", a[n-1], a[n-2], a[n-3]);
  sum = a[n-1] + a[n-2] + a[n-3];
  printf("\n Average: %d", sum/3);
  return 0;
}
```

Output:-

Enter number of salaries: 6

Enter salaries:

1200

5000

3000

25000

7000

4500

Sorted salaries:

7000 5000 4500 3000 2500 1200

Top 3: 7000 5000 4500

Average: 5500

-
- Read all sales values
 - Sort the values using Quick sort
 - Print the sales in descending order
 - Display the top three highest sales.
 - Calculate and print the average of the top three sales.

```
3Qc - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
3Qc X
1 #include <stdio.h>
2 void quick(int a[], int l, int h)
3 { if(l >= h)
4     return;
5     int i = l, j = h, p = a[l], t;
6     while(i < j)
7     { while(a[i] <= p && i < h)
8         i++;
9         while(a[j] > p)
10             j--;
11         if(i < j)
12         { t = a[i];
13           a[i] = a[j];
14           a[j] = t;
15         }
16     }
17     t = a[l];
18     a[l] = a[j];
19     a[j] = t;
20     quick(a, l, j - 1);
21     quick(a, j + 1, h);
22 }
23 int main()
24 { int n, a[100], i, sum = 0;
25   printf("Enter number of salaries: ");
26   scanf("%d", &n);
27   printf("\nEnter salaries:\n");
28   for(i = 0; i < n; i++)
29       scanf("%d", &a[i]);
30   quick(a, 0, n - 1);
31   printf("Sorted Sales:\n");
32   for(i = n - 1; i >= 0; i--)
33       printf("%d ", a[i]);
34   printf("\nTop 3: %d %d %d", a[n-1], a[n-2], a[n-3]);
35   sum = a[n-1] + a[n-2] + a[n-3];
36   printf("\nAverage: %d", sum / 3);
37   return 0;
38 }
```

Enter number of salaries: 6

Enter salaries:

1200
5000
3000
2500
7000
4500

Sorted Sales:

7000 5000 4500 3000 2500 1200

Top 3: 7000 5000 4500

Average: 5500

Process returned 0 (0x0) execution time : 18.970 s

Press any key to continue.